Фундаментальные ограничения распределённых систем

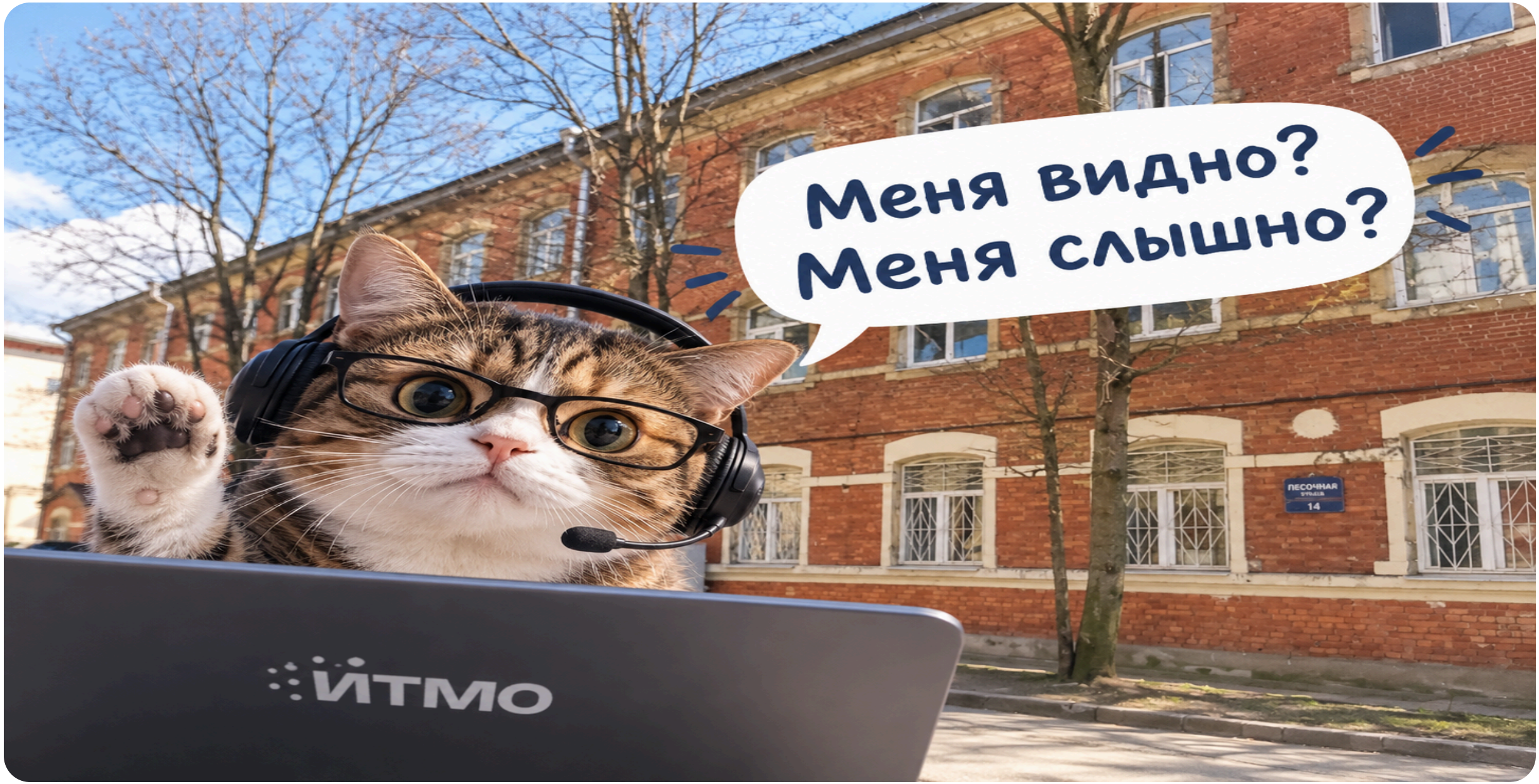
Семён Мартынов, Кирилл Дикалин, Георгий Семёнов

ИНСТИТУТ ПРИКЛАДНЫХ КОМПЬЮТЕРНЫХ НАУК

Национальный исследовательский университет ИТМО

22 октября 2026

Проверка связи



Цели лекции

1. **Объяснять**, почему масштабирование и отказоустойчивость превращают highload-систему в распределённую, и **оценивать** цену сетевого перехода.
2. **Различать** репликацию с одним и несколькими лидерами, синхронную и асинхронную, и **называть** цену каждой: задержку, потерю данных, устаревшие чтения.
3. **Формулировать** CAP и PACELC точно и **применять** их к выбору поведения системы при разделении сети и в нормальном режиме.
4. **Считать** кворумы и **объяснять**, что даёт консенсус, сколько он стоит и почему Paxos и Raft изучают отдельно.

Вопрос лекции:
за что именно мы платим, когда данные живут на нескольких узлах, и почему эту цену нельзя обнулить?

План лекции

1. Почему highload-система становится распределённой: секционирование, репликация, цена сети
2. Репликация: лидер и реплики, синхронность, отставание, несколько лидеров
3. CAP и PACELC: поведение при разделении сети и в нормальном режиме
4. Кворум и консенсус: цена согласования, Paxos и Raft

Загадка

Система: **лидер** БД и **две реплики** для чтения, балансировщик отправляет `SELECT` на любую реплику.

Пользователь нажимает «Отправить»: `INSERT` подтверждён, ответ `200 OK`. Страница перезагружается, и комментария **нет**. Через пару секунд он появляется.

Все узлы здоровы, ошибок в логах нет, метрики в норме.

> Баг? Нет: система работает **ровно так, как спроектирована**.
> Разгадка в 4.2, а общая причина в 4.1 и 4.3.

4.1 Три причины выйти за пределы одного узла

Причина	Что даёт распределение	За что платим
Ёмкость	данные и нагрузка не помещаются в один узел (у scale up есть потолок, лекция 3)	данные нужно разделить и снова собрать
Отказоустойчивость	отказ одного узла не останавливает сервис	копии нужно держать согласованными
География	данные ближе к пользователю, меньше задержка чтения	дальние копии: **физика** задержки (см. ниже)

- Stateless-слой масштабируется легко (лекция 3): трудности сдвинуты в **stateful-слой**, где данные должны быть согласованы
- Цена выхода за один узел не только деньги за железо: исчезает простая модель «одна память, один порядок событий»

4.1 Секционирование и репликация



- **Секционирование** (partitioning, sharding): разные данные на разных узлах. Растут ёмкость и пропускная способность **записи**; отказ узла теряет **часть** данных (лекция 5)
- **Репликация** (replication): одни и те же данные на нескольких узлах. Растут отказоустойчивость и пропускная способность **чтения**; запись не ускоряется, а копии нужно синхронизировать
- Реальные системы **комбинируют**: у каждой секции свои реплики. Эта лекция о репликации и её цене

4.1 Доступность: арифметика реплик

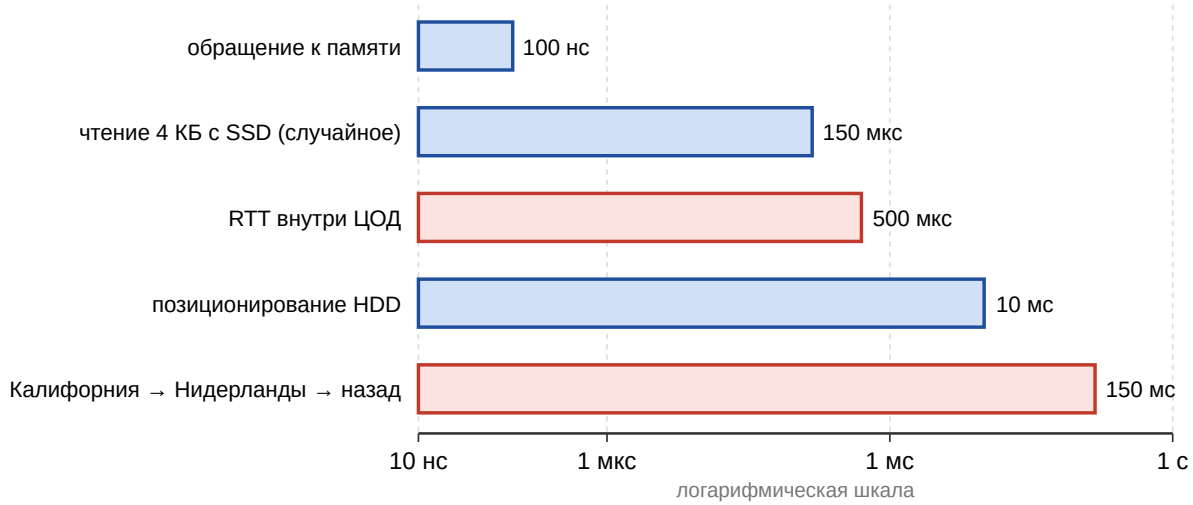
Узел доступен с вероятностью a, отказы **независимы**. Если достаточно **одной** живой копии из n:

$$A_n = 1 - (1 - a)^n$$

n при $a = 0.999$	1	2	3
Доступность	99.9%	99.9999%	99.9999999%
Простой в год	≈ 8.8 ч	≈ 32 с	≈ 0.03 с

- Формула **оптимистична**: отказы коррелируют (общая стойка, зона, релиз, баг, конфигурация)
- Она предполагает, что **любая** копия может обслужить запрос, то есть копиям не нужно «договариваться». Если договариваться нужно, например, большинству, формула другая (4.4)

4.1 Сетевой переход: порядки величин



- Красным показаны **сетевые** операции. RTT внутри ЦОД ≈ **5000** обращений к памяти, а межконтинентальный круг ≈ **300** RTT внутри ЦОД
- Железо с тех пор ускорилось, а **соотношения** остались похожими

J. Dean, «Latency Numbers Every Programmer Should Know», значения приблизительные (~2012).

4.1 Физический предел задержки

- Свет в оптоволокне ≈ $\frac{2}{3}c \approx 200\,000$ км/с, то есть **5 мкс на километр** в одну сторону
- $RTT_{min} \approx \frac{2d}{v}$: на каждые **100 км** между узлами не меньше **1 мс** на круг

Расстояние d	1 км (один ЦОД)	100 км	1 000 км	10 000 км
RTT_{min}	10 мкс	1 мс	10 мс	100 мс

- Пример: Нью-Йорк — Лондон ≈ 5 570 км, $RTT_{min} \approx 56$ мс даже по идеальной прямой. Реальные маршруты длиннее, плюс маршрутизаторы и очереди

Задержку можно уменьшить кодом, но не скоростью света. Каждый **синхронный** шаг между узлами стоит не меньше одного RTT: это нижняя граница, а не оценка.

4.1 Вызов по сети не равен вызову функции

	Локальный вызов	Вызов по сети
Результат	есть всегда	ответ может **не прийти**
Отказ	всё или ничего	**частичный**: запрос выполнен, ответ потерян
Время	наносекунды, стабильно	миллисекунды, **плавающее**, тяжёлый хвост
Порядок	один	сообщения задерживаются, дублируются, **переупорядочиваются**

- Таймаут не отвечает на вопрос «выполнилась ли операция?». Отличить **упавший** узел от **медленного** нельзя
- Отсюда идемпотентность и ретраи с jitter (лекция 7), а в этой лекции сам смысл «согласованности»
- «Восемь заблуждений распределённых вычислений» (Дойч, Гослинг и др.): сеть надёжна, задержка нулевая, пропускная способность бесконечна и так далее. Каждое превращается в инцидент

4.2 Что и зачем мы копируем

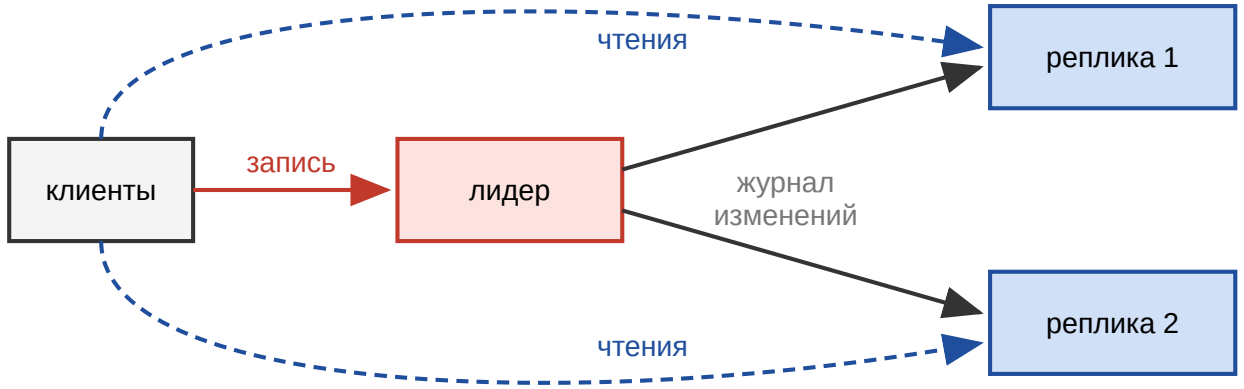
Зачем: отказоустойчивость, чтение с нескольких копий, чтение с ближайшей копии.

Что копируем: обычно не «состояние целиком», а **журнал изменений**.

- Реплика применяет те же изменения в том же порядке и получает то же состояние (идея реплицируемого автомата, *state machine replication*)
- Способы доставки: пересылка журнала предзаписи (WAL), логические изменения строк, повтор команд. Последнее требует детерминизма: `NOW()` и `RANDOM()` на двух узлах дадут разное
- Журнал — это **очередь** (лекция 2): у него есть интенсивность поступления, скорость применения и **отставание**

Главный вопрос репликации:
когда и **кому** лидер сообщает «записано»?

4.2 Лидер и реплики (leader-follower)



- Все записи идут через **одного лидера**: это единая точка упорядочения, порядок записей один у всех
- Реплики читают журнал лидера и применяют его; с них можно **читать**
- При отказе лидера одна из реплик **становится** лидером (failover)
- Лидер — последовательная часть системы: это α из USL (лекция 3)

4.2 Что масштабируется, а что нет

Допущения: ёмкость узла μ, чтение и запись стоят одинаково, каждая запись применяется на **всех** n узлах:

$$X_{read}(n) = n\,(\mu - W), \qquad X_{write} \le \mu \text{ при любом } n$$

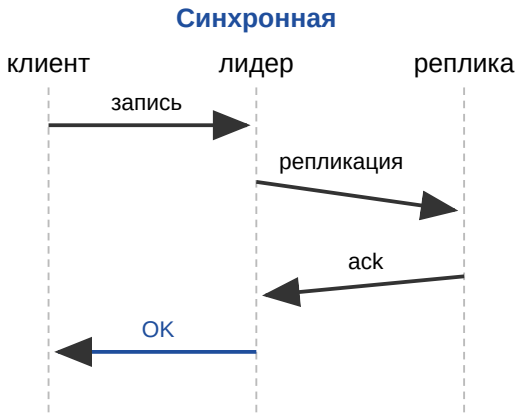
Ёмкость на чтение (оп/с) при $\mu = 10\,000$, W – интенсивность записи:

W	$n = 1$	$n = 3$	$n = 5$
2 000	8 000	24 000	40 000
5 000	5 000	15 000	25 000
9 000	1 000	3 000	5 000

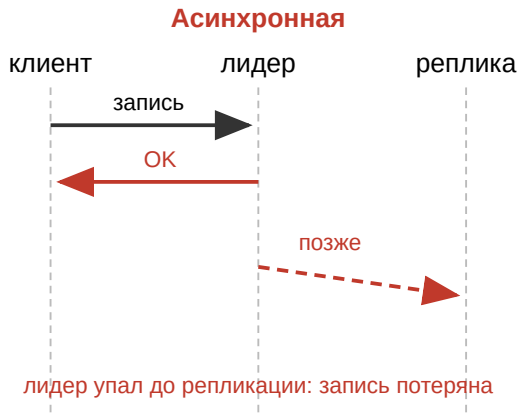
- Реплики масштабируют **чтение**, но не **запись**: каждая копия обязана применить каждую запись
- Чем выше доля записей, тем меньше толку: при $W \to \mu$ ёмкость на чтение стремится к нулю при любом n
- Модель упрощённая: применение чужой записи на реплике обычно дешевле исходного выполнения, но не бесплатно

Масштабировать **запись** нужно секциями (лекция 5), а не репликами.

4.2 Синхронная и асинхронная репликация



«OK» после подтверждения реплики



«OK» сразу, реплика догоняет позже

- **Синхронная**: лидер отвечает «OK» после подтверждения реплики. Задержка записи $t \approx t_{\text{лидер}} + RTT + t_{\text{реплика}}$, а RTT — нижняя граница из 4.1
- **Асинхронная**: «OK» сразу, реплика догоняет **позже**. Быстро, но подтверждённая запись может существовать только на лидере

4.2 Синхронная или асинхронная: цена

	Синхронная	Асинхронная
Задержка записи	$+RTT$ до реплики и её запись на диск	как у одного узла
Отказ лидера	подтверждённые записи **сохранены** на реплике	подтверждённые записи **могут пропасть**
Реплика недоступна или медленна	запись **останавливается или тормозит**	запись не страдает
Чтение с реплики	свежее, только если лидер ждёт **применения**	**отстаёт** (replication lag)
Типичное применение	потеря недопустима, реплика рядом	реплика далеко, важна задержка

- Синхронность на **все** n реплик: при независимых отказах доступность записи $A_{write} = a^n$, то есть с ростом n она **падает**

Поэтому на практике используют **полусинхронную** схему (следующий слайд) и кворум (4.4).

4.2 Полусинхронная схема и кворумная фиксация

- **Полусинхронная** схема (semi-synchronous): синхронно ждём **одну** реплику или k из m, остальные асинхронно. Подтверждённую запись потеряем, только потеряв **и** лидера, и синхронную реплику
- Пример, PostgreSQL: параметр `synchronous_commit` выбирает, чего ждать от реплики

`synchronous_commit`	Лидер ждёт от реплики
`remote_write`	запись получена и передана ОС, без сброса на диск
`on`	запись сброшена на диск
`remote_apply`	запись **применена**, то есть видна читателям реплики

- `synchronous_standby_names = 'ANY 1 (s1, s2, s3)'`: достаточно **любой одной** из трёх (кворумная фиксация); `'ANY 2 (s1, s2, s3)'` ждёт двух
- Каждая следующая ступень добавляет ожидание. Выбрать, сколько подтверждений ждать, значит выбрать точку на шкале «задержка — риск потери»

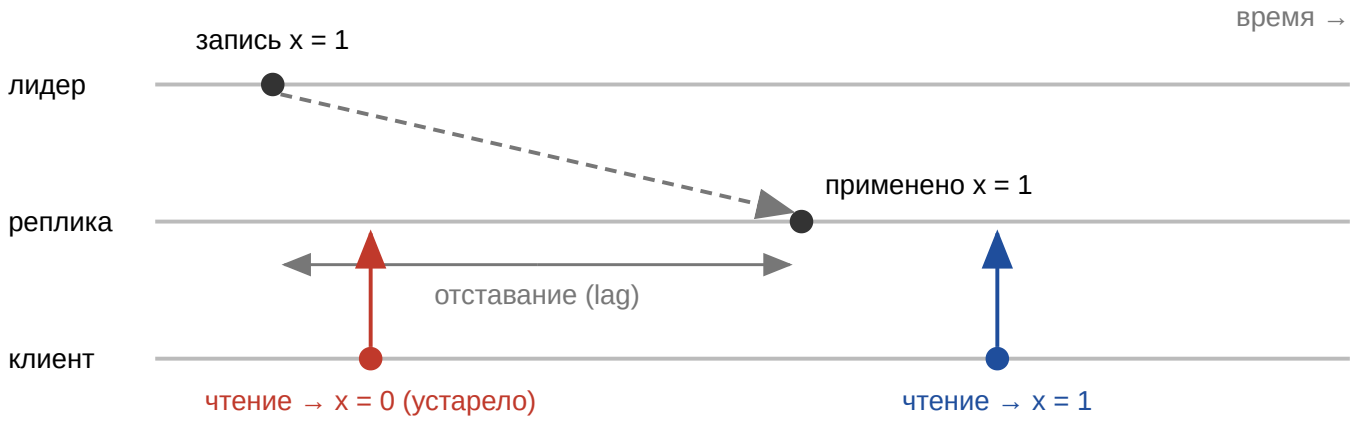
PostgreSQL Documentation, «High Availability, Load Balancing, and Replication». Пример одной СУБД, идея универсальна.

4.2 Отказ лидера: failover

Проблема	Почему возникает
Потеря подтверждённых записей	при асинхронной репликации у новой реплики-лидера нет хвоста журнала
Два лидера (**split-brain**)	старый лидер «жив», но отрезан от сети: оба принимают записи
Ложное срабатывание	таймаут слишком короткий: под нагрузкой (пик, пауза GC) лидер «молчит», его заменяют, нагрузка растёт
Долгий простой	таймаут слишком длинный: запись недоступна, пока ждём

- Нужно **ограждение** (fencing): старый лидер не должен суметь записать
- Но «кто сейчас лидер?» снова вопрос согласия узлов, то есть **консенсус** (4.4)

4.2 Устаревшие чтения и отставание реплик



- Реплика применяет журнал с **отставанием** (replication lag), оно меняется от запроса к запросу
- Журнал — очередь (лекция 2): записи приходят с интенсивностью λ_w, реплика применяет со скоростью μ_{apply}. При $\rho = \lambda_w / \mu_{apply} \to 1$ отставание растёт нелинейно, как «хоккейная клюшка»
- Типичные причины: пик записей, тяжёлая транзакция, **последовательное** применение на реплике при параллельной записи на лидере, сеть между регионами

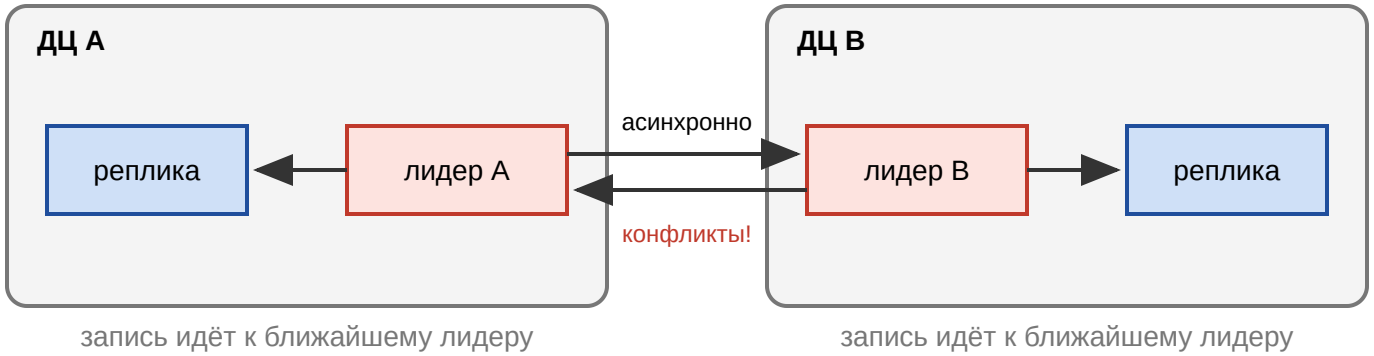
4.2 Разгадка и гарантии для читателя

Разгадка: `INSERT` ушёл на лидера, а `SELECT` попал на реплику, которая ещё не применила запись. Нарушена гарантия **«читай свои записи»**.

Гарантия	Нарушение	Как обеспечить, и цена
Читай свои записи (read-your-writes)	своя запись не видна	читать «своё» с лидера или после метки позиции журнала. Цена: нагрузка на лидера, сложнее маршрутизация
Монотонные чтения (monotonic reads)	второе чтение показывает **более старые** данные, чем первое	привязать пользователя к одной реплике. Цена: перекос нагрузки, переезд при отказе
Согласованный префикс (consistent prefix)	ответ виден раньше вопроса	связанные записи держать в одной секции или в одном журнале. Цена: ограничения модели данных

Терминология: M. Kleppmann, «Designing Data-Intensive Applications», гл. 5.

4.2 Несколько лидеров (multi-leader): обзор



- Каждый ДЦ принимает записи у **своего** лидера: запись локальна, ДЦ переживает изоляцию от соседа
- Цена: одну и ту же запись можно изменить в двух местах одновременно, **конфликты** обнаруживаются постфактум
- Разрешение: **избегать** (ключ закреплён за одним лидером); «последняя запись побеждает» (LWW), просто, но **молча теряет** записи и зависит от часов; **слияние** (CRDT); решение приложения
- Применяют: многорегиональные системы, офлайн-клиенты, совместное редактирование. Если можно обойтись одним лидером, обойтись

4.2 Мостики: репликация повсюду

Область	Что реплицируется	Связь с курсом
Архитектура ЭВМ	строки кэша между ядрами	протокол когерентности — репликация с инвалидацией; цена — β в USL, false sharing (лекция 10)
Git	репозиторий	несколько «лидеров»: ветки расходятся, конфликты решаются слиянием
DNS, CDN, кэши	записи и содержимое	реплика с TTL: окно устаревания задано явно (лекция 5)
Брокеры сообщений	журнал секции	лидер секции и реплики (лекция 6)

Один и тот же вопрос в разных масштабах: **сколько ждать согласия копий и что делать, пока оно не достигнуто**.

4.3 CAP: формулировка

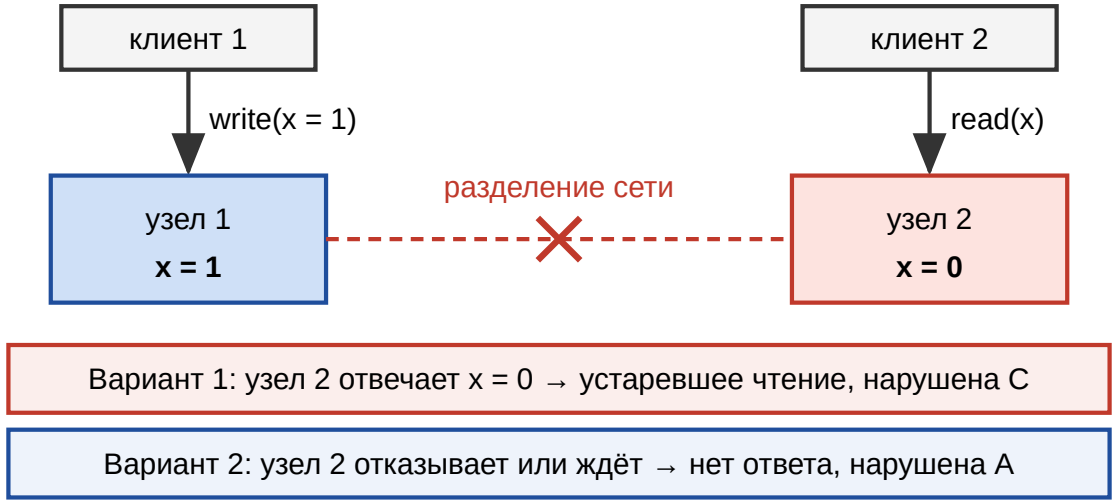
Свойство	Точный смысл
C: согласованность (consistency)	**линеаризуемость**: система ведёт себя так, будто копия **одна**, и чтение видит последнюю завершённую запись
A: доступность (availability)	**каждый** запрос к неотказавшему узлу получает ответ, а не ошибку и не вечное ожидание
P: устойчивость к разделению (partition tolerance)	система продолжает работать, даже если сеть **теряет** произвольные сообщения между узлами

Теорема CAP: в асинхронной сети нельзя одновременно обеспечить C, A и P.

- Гипотеза: Brewer, PODC 2000 (доклад). Формализация и доказательство: Gilbert, Lynch, 2002

Gilbert, Lynch, «Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services», ACM SIGACT News 33(2), 2002.

4.3 CAP: доказательство в одной картинке



- Узел 2 не может узнать о записи: сообщение не дошло. Любой его ответ — выбор между **C** и **A**
- Доказательство занимает страницу, потому что утверждение простое: нельзя знать то, о чём тебе не сообщили

4.3 P не выбор: выбирают между C и A

- Разделения случаются: отказ коммутатора, перегруженный канал, долгая пауза GC. Снаружи они **неотличимы** от потери связи (4.1)
- «CA»-системы среди распределённых нет: у одного узла разделения нет, а у нескольких оно **произойдёт**
- Выбор делается **на момент разделения**: что делает система, когда узлы не видят друг друга

Поведение	При разделении	Пример
CP: сохранить C	сторона без большинства **отказывает** (ошибка или ожидание)	etcd, ZooKeeper: запись требует большинства
AP: сохранить A	обе стороны **продолжают** принимать записи, расхождения сводят потом	Dynamo: «всегда доступна для записи»

Пока разделения нет, выбора между C и A нет. Но есть выбор между C и **задержкой**: см. PACELC.

4.3 Чего CAP не говорит

- **Ничего про задержку:** система может быть «CP» и медленной, «AP» и быстрой, и наоборот
- **Узкие определения:** C — именно линеаризуемость; A — ответ от **каждого** неотказавшего узла, что строже практической «доступности 99.9%»; единственный сбой — разделение сети
- **«CP» и «AP» — не свойства продукта:** многие системы настраиваются, а метка не говорит, какая гарантия и на каких операциях
- **Согласованность не бинарна**, это шкала (следующий слайд)

CAP — ограничение на **поведение при разделении**, а не классификатор баз данных.

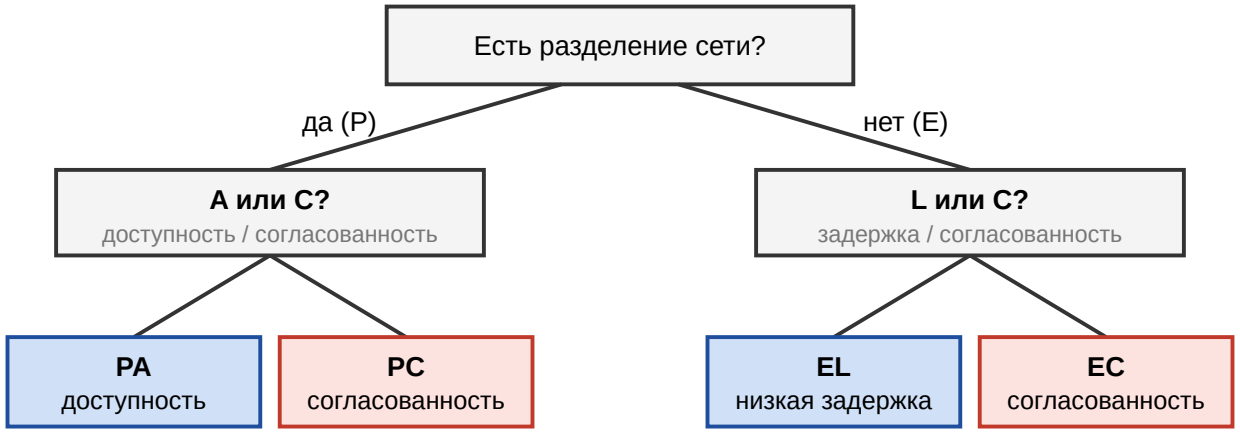
M. Kleppmann, «A Critique of the CAP Theorem» (2015) и «Please stop calling databases CP or AP» (2015); E. Brewer, «CAP Twelve Years Later», IEEE Computer, 2012.

4.3 Шкала согласованности: архитектурный выбор

Модель	Что гарантирует читателю	Цена
Линеаризуемость (strong)	как с одной копией: чтение видит последнюю завершённую запись	координация на каждую операцию: задержка и недоступность при разделении
Причинная (causal)	если B зависит от A, все видят A раньше B	метаданные о зависимостях; параллельные записи можно видеть в разном порядке
Читай свои записи, монотонные чтения	гарантии для **одного** клиента (4.2)	маршрутизация, привязка клиента к реплике
Итоговая (eventual)	если записи прекратятся, копии **со временем** сойдутся	в промежутке возможны любые аномалии; нужен механизм сведения копий

- Чем сильнее модель, тем больше **координации** между узлами; координация стоит RTT и пропадает при разделении
- Есть модели слабее линеаризуемости (например, причинная), совместимые с доступностью при разделении: между «всё» и «ничего» есть середина

4.3 PACELC: что происходит без разделения



PACELC: if Partition then (A or C), Else (L or C)

- Разделения случаются время от времени, а запросы идут каждую миллисекунду. Даже при здоровой сети согласованность стоит ожидания: чтобы копии совпадали, нужно дождаться других узлов
- **PACELC** (Abadi, 2012): *при* разделении выбор между доступностью (A) и согласованностью (C), *иначе* (E) между **задержкой** (L) и согласованностью (C)
- Вторая половина — то, что мы платим **за каждый запрос**, а не раз в год

4.3 Системы в координатах PACELC

Класс	Поведение	Примеры (Abadi, 2012)
PA/EL	при разделении доступность, в норме задержка	Dynamo, Cassandra, Riak
PA/EC	при разделении доступность, в норме согласованность	MongoDB
PC/EL	при разделении согласованность, в норме задержка	PNUTS
PC/EC	согласованность всегда	VoltDB / H-Store, Megastore

Классификация относится к **конфигурации по умолчанию** систем 2012 года. Версии и настройки (уровни согласованности, подтверждения записи) двигают систему по таблице: смотрите документацию **своей** версии, а не эту таблицу.

4.3 Цена согласованности в числах

Лидер в регионе A, реплика B с $RTT = 20$ мс, реплика C с $RTT = 150$ мс (условные значения). Добавка к задержке записи:

Схема	Чего ждём	Добавка
Асинхронная	никого	≈ 0
Кворум 2 из 3 (лидер и ближайшая реплика)	реплику B	≈ 20 мс
Синхронно на все	реплику C	≈ 150 мс

- Синхронность на **все** реплики делает задержку равной RTT до **самой дальней**
- Кворум платит RTT до **ближайшего большинства**: дальняя реплика уходит с критического пути, но остаётся страховкой
- Обратная сторона: если B недоступна, кворум упирается в C, то есть **150 мс**

Убрать RTT нельзя, можно выбрать, до **кого** его платить.

4.3 Пример: Spanner платит за согласованность

- **Spanner** (Google): глобально распределённая, **синхронно реплицируемая** БД. Каждая секция (tablet) реплицируется группой Paxos, реплики могут находиться в разных ДЦ
- Гарантия: **внешняя согласованность** транзакций, то есть порядок транзакций совпадает с порядком в реальном времени
- Цена: запись ждёт кворума Paxos между ДЦ (это RTT), а фиксация транзакции ещё и **неопределённости часов** (commit wait, механизм TrueTime)
- Для часов Google использует специальное оборудование (GPS и атомные часы в ЦОД)

Сильная согласованность в глобальном масштабе возможна, но платить за неё приходится **задержкой** и инфраструктурой. Это осознанный выбор для данных, где расхождение дороже ожидания.

Corbett et al., «Spanner: Google's Globally-Distributed Database», OSDI 2012.

4.3 Как выбирать: по данным, а не по системе

Данные или операция	Что нужно	Почему
Баланс, остатки, уникальность (логин, номер заказа)	**сильная** согласованность	гонка даёт **невосстановимую** ошибку
Настройки пользователя	«читай свои записи»	пользователь должен видеть своё, остальным можно опаздывать
Лента, счётчики просмотров, лайки	**итоговая**	несколько секунд устаревания не дефект
Конфигурация, выбор лидера, блокировки	**линеаризуемость**, малый объём	ошибка ломает всю систему, а данных мало и цену можно заплатить (семинар 4)

- Одна система смешивает режимы (как компоненты в 3.1): строго для **малого и критичного**, итогово для **большого и терпимого**
- Выбор формализуется через SLO, следующий слайд

4.3 Выбор из SLO: физика отсекает варианты

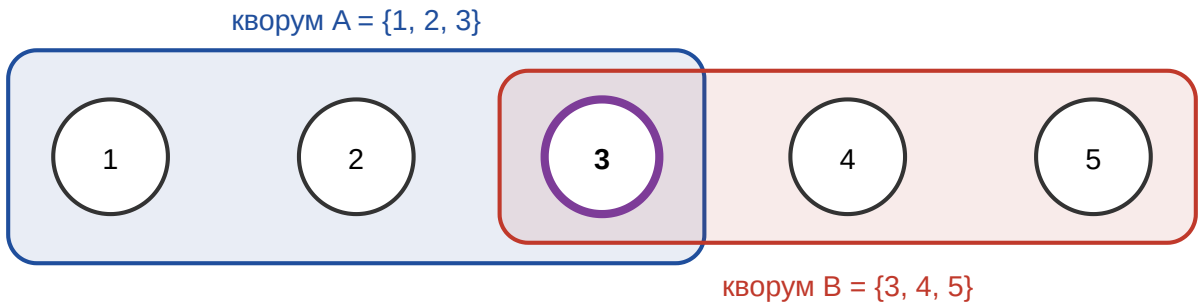
Дано: SLO записи $p_{99} \leq 100$ мс. Лидер и клиенты в Европе. Для устойчивости к потере региона нужна копия в США, $RTT \approx 90$ мс (условно). Допустимая потеря данных (**RPO**) задана заказчиком.

Вариант	Задержка записи	Потеря зоны	Потеря региона
1. Синхронно в США	≥ 90 мс; на всё остальное остаётся ≤ 10 мс, SLO **практически не выдержать**	без потери	без потери
2. Асинхронно в США	как у одного узла	потеря хвоста журнала	потеря хвоста журнала
3. Синхронно в другую зону (≈ 1–2 мс), асинхронно в США	единицы мс, SLO **выполнен**	без потери	потеря хвоста журнала

- При $RTT \approx 90$ мс на диск, обработку и джиттер сети остаётся меньше 10 мс, поэтому требование $p_{99} \leq 100$ мс **практически исключает** синхронную репликацию между континентами. Остаётся выбрать, **что потерять** при катастрофе
- Вариант 3 — типичный компромисс: быстро, переживает потерю зоны, а размер потерь при потере региона ограничен отставанием

SLO (лекция 1) превращает спор «C или A» в арифметику.

4.4 Кворум: определение



- **Кворум** — подмножество узлов, чьего согласия достаточно для решения
- **Большинство:** $\lfloor n/2 \rfloor + 1$ узлов. Любые два большинства **пересекаются** хотя бы в одном узле, потому что $|Q_1| + |Q_2| > n$
- Общий узел «помнит» прошлое решение: новое решение не может его **забыть**, и двух «истин» не возникает

4.4 Размер кластера и допустимые отказы

n	1	2	3	4	5	6	7
Большинство	1	2	2	3	3	4	4
Допустимо отказов f	0	0	1	1	2	2	3

- $f = \lfloor (n-1)/2 \rfloor$, то есть для f отказов нужно $n = 2f + 1$ узлов
- **Чётное** n отказоустойчивости не добавляет: 4 узла терпят столько же отказов, сколько 3, а подтверждений нужно больше
- Два узла хуже одного: любой отказ лишает большинства, и «кластер» из двух узлов доступен **хуже**, чем один узел
- Больше узлов — **медленнее** запись: данные идут на большее число машин. Поэтому etcd рекомендует 3, 5 или 7 узлов

etcd Documentation, FAQ: «Why an odd number of cluster members?», «What is maximum cluster size?».

4.4 Кворумные чтения и записи: R + W > N

Репликация **без лидера** (как в Dynamo): N реплик, запись подтверждена W узлами, чтение опрашивает R узлов и берёт самую свежую версию.

$\boxed{R + W > N}$ означает, что узлы чтения **пересекаются** с узлами последней записи.

(N, W, R)	Свойства
$(3, 2, 2)$	сбалансировано: терпит 1 отказ и на чтение, и на запись (типичная настройка Dynamo)
$(3, 3, 1)$	быстрое чтение, а запись требует **всех** узлов: недоступна при одном отказе
$(3, 1, 3)$	быстрая запись, а чтение требует всех узлов
$(3, 1, 1)$	$R + W \leq N$: пересечения нет, возможна только **итоговая** согласованность

Пересечение **не равно** линеаризуемости: одновременные записи, частично применённые записи и разрешение конфликтов по часам могут вернуть устаревшее. При отказах Dynamo пишет на **соседние** узлы («нестрогий» кворум, hinted handoff), и пересечение уже не гарантировано.

4.4 Кворум и хвост задержки: ждём k-го из n

Операция ждёт не **всех**, а k-го по скорости ответа из n: медленный узел уходит с критического пути. Пусть каждый ответ медленнее порога T с вероятностью 1%, независимо:

Ждём	Вероятность, что операция медленнее T
всех 3	$1 - 0.99^3 \approx$ **3.0%**
2 из 3	$3 \cdot 0.01^2 \cdot 0.99 + 0.01^3 \approx$ **0.03%**
всех 5	$1 - 0.99^5 \approx$ **4.9%**
3 из 5	$\approx$ **0.001%**

- Кворум **лечит хвост** (идея Dean, Barroso, 2013, лекция 3), но платит **трафиком и нагрузкой**: запись уходит на все n узлов, а ждёт ответа лишь от k
- Независимость — допущение: коррелированные сбои (общая сеть, одновременные паузы GC) съедают выигрыш

4.4 Цена кворума по доступности

Узел доступен с вероятностью $a = 0.999$, отказы независимы. Вероятность недоступности и простой в год:

n	Достаточно одной копии	Нужно большинство
1	10^{-3} (≈ 8.8 ч)	10^{-3} (≈ 8.8 ч)
2	10^{-6} (≈ 32 с)	$2 \cdot 10^{-3}$ (≈ 17.5 ч)
3	10^{-9} (≈ 0.03 с)	$3 \cdot 10^{-6}$ (≈ 95 с)
5	10^{-15} (≈ 0)	10^{-8} (≈ 0.3 с)

- «Достаточно одной копии» даёт максимум доступности, но **без согласия**: копии могут разойтись (AP-сторона CAP)
- Требование большинства стоит при $n = 3$ **трёх порядков** доступности, но даёт единую «истину» (CP-сторона CAP)
- Рост n улучшает доступность большинства, но замедляет запись (предыдущие слайды)

Так выглядит цена выбора «C вместо A». Модель упрощённая: она считает независимые отказы узлов, а не разделения сети.

4.4 Консенсус: задача

- Узлам нужно **договориться об одном значении** (кто лидер? какая следующая запись журнала? какая конфигурация?), даже если часть узлов отказывает, а сообщения задерживаются
- Свойства: **согласие** (agreement) — все решившие выбрали одно и то же; **допустимость** (validity) — решено то, что кто-то предложил; **завершение** (termination) — неотказавшие узлы в итоге решают
- Повторяя консенсус для каждой записи, получаем **общий журнал**. Одинаковые автоматы, применяющие один журнал, приходят к одному состоянию (*state machine replication*)
- Где нужен: выбор лидера, блокировки и аренды (lease), конфигурация, упорядочивание записей в реплицируемой БД

> Консенсус превращает «несколько узлов» в «одну логическую копию».

4.4 Почему консенсус нельзя просто сделать

- **FLP** (Fischer, Lynch, Paterson, 1985): в асинхронной системе **ни один** детерминированный протокол не гарантирует завершение консенсуса, если может отказать даже **один** процесс
- Причина знакома из 4.1: упавший узел **неотличим** от медленного
- Практика: Paxos и Raft **безопасны всегда** (при отказах остановкой, без злонамеренных узлов никогда не решат разное), а **прогресс** обеспечивают при «достаточно синхронной» сети: таймауты и случайные задержки перед выборами
- Raft требует $\mathrm{broadcastTime} \ll \mathrm{electionTimeout} \ll \mathrm{MTBF}$: таймаут выборов **много больше** RTT, а отказы редки
- При разделении сторона без большинства **не продвигается**: это CP из 4.3

Fischer, Lynch, Paterson, «Impossibility of Distributed Consensus with One Faulty Process», JACM 32(2), 1985; Ongaro, Ousterhout, USENIX ATC, 2014.

4.4 Paxos и Raft: зачем нужны

	Paxos	Raft
Автор, год	Lamport; придуман около 1990, опубликован в 1998 («The Part-Time Parliament»), проще изложен в 2001 («Paxos Made Simple»)	Ongaro, Ousterhout, 2014 (USENIX ATC)
Идея	согласие большинства вокруг нумерованных предложений	**сильный лидер**; выборы, репликация журнала и безопасность разделены
Главная цель	корректность	**понятность**, при результате, эквивалентном multi-Paxos, и сопоставимой эффективности
Где встречается	Google: Chubby, Spanner	etcd, Consul

- Общее: большинство из $n = 2f + 1$ узлов, терпят f отказов, на каждое решение нужен как минимум один RTT до большинства

4.4 Raft: репликация журнала (интуиция)



- Запись идёт **через лидера** и считается зафиксированной, когда лежит у **большинства**; отставшая реплика догоняет позже
- Для $n = 2f + 1$ лидеру нужны подтверждения f реплик: $t_{commit} \gtrsim RTT_{(f)} + t_{fsync}$, где $RTT_{(f)}$ — f-й по скорости ответ (оценка)

4.4 Что даёт консенсус и чего стоит

Даёт	Стоит
единый линеаризуемый журнал, нет split-brain	не меньше одного RTT до большинства на **каждую** запись и запись на диск
терпит f отказов из $2f + 1$ узлов	запись идёт через **одного лидера**: пропускная способность не растёт с n (α в USL, лекция 3)
автоматическая смена лидера	меньшинство при разделении **недоступно**; при смене лидера запись на паузе
чёткие гарантии	сложность реализации и эксплуатации

Единый консенсус-кластер (etcd, ZooKeeper) держат для **небольшого критичного состояния**: кто лидер, конфигурация, блокировки. Большие объёмы тоже можно реплицировать консенсусом, но по секциям (так устроен Spanner), и тогда цена RTT и лидера платится на каждую запись каждой секции. Готовый компонент берут, а не пишут свой.

4.4 Почему Paxos и Raft изучают отдельно

В этой лекции

- зачем нужен консенсус
- что он гарантирует
- сколько он стоит: RTT до большинства, один лидер, недоступность меньшинства
- как это увидеть в измерениях (семинар 4)

В курсе по распределённым системам

- доказательства безопасности и живости
- смена состава кластера (joint consensus в Raft)
- сжатие журнала и снимки состояния
- линеаризуемые чтения
- византийские отказы

Мы используем консенсус как **чёрный ящик с известной ценой** и не дублируем отдельный курс.

Чек-лист инженера

1. Решите, **зачем** выходить за один узел: ёмкость (секции), отказоустойчивость или география (реплики)
2. Посчитайте **нижнюю границу задержки** $RTT_{min} = 2d/v$ для каждого синхронного шага между узлами
3. Для каждой реплики выберите **синхронно, полусинхронно или асинхронно** и запишите допустимую потерю данных
4. Выбирайте согласованность **по данным**, а не по системе; для читателей обеспечьте «читай свои записи» и монотонность там, где это нужно
5. Измеряйте **отставание реплик** и не читайте с реплики то, что нельзя читать устаревшим
6. Для кворума берите **нечётное** n (3 или 5) и помните: рост n замедляет запись
7. Консенсус применяйте для **небольшого критичного состояния**; для больших объёмов считайте его цену на каждую секцию. Берите готовый компонент

Заключение

Идея	Суть
Распределённость	секции дают ёмкость, реплики дают отказоустойчивость; цена сети: $RTT_{min} = 2d/v$
Репликация	лидер и реплики масштабируют чтение, но не запись; у записей один порядок
Синхронность	синхронно: $+RTT$, но без потерь; асинхронно: быстро, но возможны потери и отставание
Отставание	устаревшие чтения; нужны гарантии «читай свои записи» и монотонные чтения
CAP	при **разделении** выбор между C (линеаризуемость) и A; P не выбирают
PACELC	без разделения выбор между **задержкой** и согласованностью
Кворум	любые два большинства $\lfloor n/2 \rfloor + 1$ пересекаются; $R + W > N$; ждём k-го из n
Консенсус	общий журнал и лидер; не меньше RTT до большинства на запись; Paxos и Raft — отдельный курс

Семинар 4: etcd и ZooKeeper, **цена координации** в измерениях. Следующая лекция: **архитектура высоконагруженных систем**, то есть балансировка, кэширование, очереди и секционирование.

Список источников

- Brewer. *Towards Robust Distributed Systems* (PODC 2000, keynote)
- Gilbert, Lynch. *Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services* (ACM SIGACT News 33(2), 2002)
- Brewer. *CAP Twelve Years Later: How the «Rules» Have Changed* (IEEE Computer 45(2), 2012)
- Abadi. *Consistency Tradeoffs in Modern Distributed Database System Design: CAP is only part of the story* (IEEE Computer 45(2), 2012)
- Kleppmann. *A Critique of the CAP Theorem* (arXiv:1509.05393, 2015); *Please stop calling databases CP or AP* (2015)
- Kleppmann. *Designing Data-Intensive Applications* (O'Reilly, 2017): гл. 5 «Репликация», гл. 9 «Согласованность и консенсус»
- DeCandia et al. *Dynamo: Amazon's Highly Available Key-value Store* (SOSP 2007)
- Fischer, Lynch, Paterson. *Impossibility of Distributed Consensus with One Faulty Process* (JACM 32(2), 1985)
- Lamport. *The Part-Time Parliament* (ACM TOCS 16(2), 1998); *Paxos Made Simple* (ACM SIGACT News 32(4), 2001)
- Ongaro, Ousterhout. *In Search of an Understandable Consensus Algorithm* (USENIX ATC, 2014)
- Corbett et al. *Spanner: Google's Globally-Distributed Database* (OSDI 2012)
- Dean, Barroso. *The Tail at Scale* (Communications of the ACM, 2013)
- Dean. *Latency Numbers Every Programmer Should Know* (~2012)
- PostgreSQL Documentation. *High Availability, Load Balancing, and Replication* (`synchronous_commit`, `synchronous_standby_names`)
- etcd Documentation. *FAQ* (размер кластера, кворум)

Обратная связь

Анонимная форма